Table of Contents

1.0 Introduction **2**
1.1 Background 2
1.2 Problem Statement and Characteristics 2

2.0 Data **2**
2.1 Data Sources 2

3.0 Method Overview **2**
3.1 Overview 2

4.0 Exploratory Data Analysis **3**
4.1 Data Preprocessing 3
4.2 Data Visualization 3

5.0 Model **4**
5.1 Baseline Model 4
5.2 Logistic Regression Classifier 5
5.3 Random Forest Classifier 5
5.4 Decision Tree Classifier 6
5.5 K-Nearest Neighbors Classifier 6
5.6 Final Model Selection 6

6.0 Simulation **7**
6.1 Job Performance Optimization Model 7
6.2 Simulation Logistic 7
6.3 Results 7
6.4 Sensitivity Analysis 8

7.0 Discussions **8**
7.1 Comparison and possible restrictions of the job satisfaction predictor 8
7.2 Comparison and possible restrictions of the competence predictor 8
7.3 Comparison and possible restrictions of the Job Performance Optimization Model 8
7.4 Possible factors not considered 9

8.0 Future Direction **9**
8.1 Future Research Direction 9

9.0 References **10**

1.0 Introduction

1.1 Background

Nowadays, machine learning is widely used in the field of human resource analytics, especially in talent acquisition. Outdated pre-screening approaches such as paper/phone screens and manual assessments are extremely slow and inefficient. By adopting machine learning technology, the speed of hiring could increase by 85% [1]. However, recruiters with no technical backgrounds are still facing great challenges when it comes to hiring IT roles. The most common way to pre-screen IT roles is to use a third-party online assessment tool, but this is time-consuming for job applicants and recruiters select applicants based barely on their performances on one task.

1.2 Problem statement

After identifying this gap in the current HR market, we want to develop an interview pre-screening application specifically for IT roles that helps recruiters predict applicants' job performances. Employers wish to recruit applicants who would obtain high job performance throughout the period of employment. However, considering that job performance is multidimensional, the team hopes to exploit the key factors that impact job performance and predict job performance by predicting those factors. Through our research, job satisfaction and technical competency may serve as primal indicators of job performance [2].

Therefore, we developed rigorous models to predict job satisfaction score and competence score of each applicant. Our goal was to predict an applicant's job performance based on a combination of predicted job satisfaction and competence scores.

Finally, we used these models to build a job performance optimization model that simulated an interview pre-screening process for a developer role.

2.0 Data

2.1 Data Source

The raw dataset comes from a recent public survey [3] taken by the StackOverflow online developer community which includes 84 attributes and 81,335 rows. It contains a comprehensive list of questions to the developers' career and personal life.

We carefully screened feature attributes and dropped those out of scope or with little correlation with the target variables. The finally adopted features are briefly explained in [Appendix 2]. Discarded features are available in [Appendix 1]. Overall, the EDA-ready data frame has 2 categorical target variables: Job Satisfaction and Competence, as well as 65 feature variables. Some presumed key variables are annual salary, education level, and years of coding experience. [Appendix 2].

3.0 Method Overview

3.1 Overview

The team conducted Exploratory Data Analysis (EDA) in cleaning and transforming the raw dataset and visualized certain statistical relationships that provided useful insights for model selections. For each target variable, the team first established a baseline model as a reference. Since our problem is a multi-class classification, we chose logistic regression classifier, random forest classifier, decision tree classifier and k-nearest neighbors classifier as four applicable models to train and test data for each target variable. The best model is selected from the four candidate models and compared against the

baseline model for each target variable. Next, the team employed simulation method to evaluate the performance of the proposed model by comparing it with the optimal solution and the random selection model.

4.0 Exploratory Data Analysis

4.1 Data preprocessing

Data preprocessing starts with parsing strings [Appendix 3]. For multiple-choice responses, semicolon-delimited strings were parsed and one-hot-encoded into binary variables.
In the handling of missing values, we first dropped the rows that contain any missing values in the two target variables. In addition, rather than removing rows with missing data in other features, we imputed them with the median values in place of missing data for numeric feature variables, and populate missing values only for those correlated categorical features with the mode from the population. [Appendix 4] Eventually, we kept 37,170 rows for the data set.
In order to parse the target variables, we converted the likert-scale job satisfaction and competence into numerical values respectively. [Appendix 5] We ascendingly aligned 0 - 4 correspondingly with very dissatisfied - very satisfied for job satisfaction; far below average - far above average for competence. Same methodologies were applied to categorical variables that contain logical increments. For example, increasing education levels were converted into likert-type numeric values. Categorical variables that contain no logically incremental responses, like various indiscriminate undergrad majors, were one-hot-encoded into binary variables.

4.2 Data Visualization

EDA starts with a bar-plot delineation of the distribution of each target variable. The job satisfaction distribution shows a left-skewed graph *(Figure 4.0)*. And the competence variable approximates a normal distribution. *(Figure 4.1)* The graphs both indicate that job satisfaction and competence are imbalanced target variables. From these two graphs, it sparks the idea of a baseline model which always predicts the mean value of the target variables from the training dataset.
The box plot *(Figure 4.2)* and the bar plot *(Figure 4.3)* both demonstrate that there is a positive correlation between job satisfaction and competence. As the competence level maximizes, the job satisfaction level also gets higher. Therefore, when doing simulation at the subsequent step, we can ensure that both target variables can be optimal.
Next, we ran a test of correlation between every single feature variable and the target variable. [Appendix 7] Annual salary is an evident indicator of job satisfaction. We plotted a lmplot *(Figure 4.4)* with the mean point and 95% CI, showing the overall positive correlative trend between annual salary and job satisfaction, while it further tells a curved nonlinear relationship which may indicate model preference on random forest and logistic regression rather than linear regression.

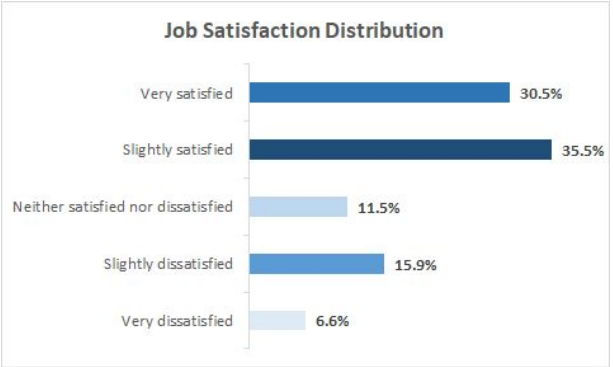


Figure 4.0. Bar-plot of job satisfaction

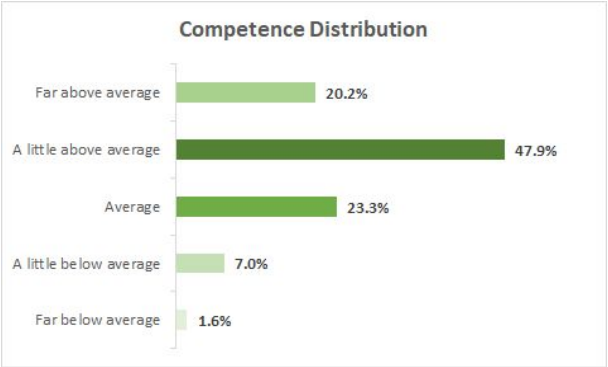


Figure 4.1. Bar-plot of competence

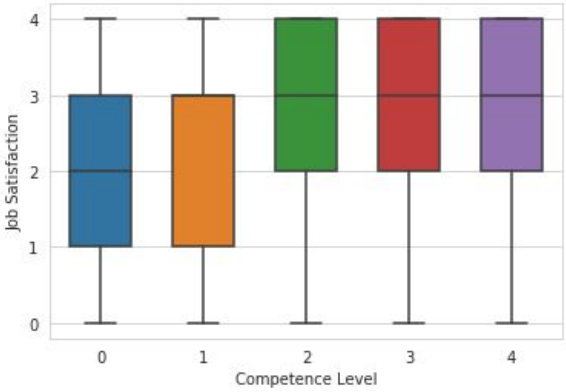


Figure 4.2 Box-plot of job satisfaction vs. competence

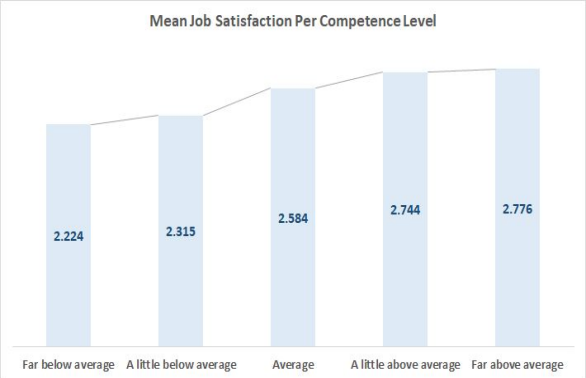


Figure 4.3 Bar-plot of mean job satisfaction per competence

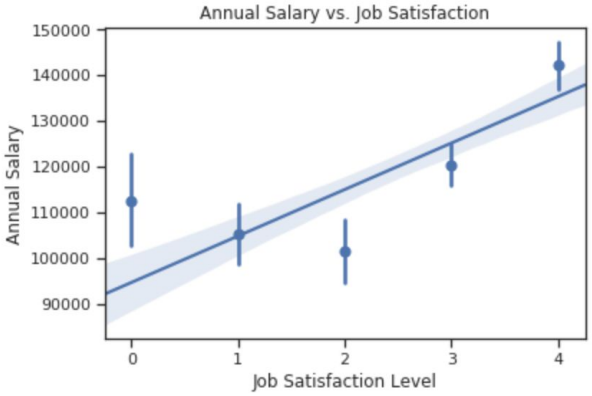


Figure 4.4 lmplot of Mean annual salary vs. Job satisfaction

5.0 Model building

5.1 Baseline Model

First of all, we split and retained 80% of the data as training dataset, and used 20% of the data for testing. The baseline model took in the values of the target variable from the training dataset and always returned the mean of that variable [Appendix 8]. The mean job satisfaction score is 3 and the mean competence score is also 3. The precision and mean absolute errors are listed below [Table 1]. Precision measures the proportion of data points predicted correctly, while mean absolute error (MAE) measures the randomness of data. For each baseline model, the precision is really close to the proportion of the largest class in the population. The MAE of the Job Satisfaction baseline model is larger than the MAE of the Competence

baseline model, indicating that Job Satisfaction is relatively more random than Competence. The MAE scores for both models are relatively high, corresponding to the imbalance of classes in both datasets.

Table 1: Baseline model performances

Target variable	Precision	Mean absolute error
Job Satisfaction	0.3558	0.9426
Competence	0.4708	0.6270

5.2 Logistic Regression Classifier

A logistic regression classifier was built for each variable with tuned hyperparameters. The precision for each model with tuned hyperparameters is listed in Table 2 [Appendix 9]. Both models performed better than the baseline models. However, we found out that MAE did not imply any characteristic of the model other than how much the prediction deviated from true values. Also, MAE is usually used to deal with continuous variables while our variables are integers.

Therefore, we decided to use another metric: % of data points predicted lower than actual values, to analyze whether the model was biased to predict higher or lower than the true values. Detailed justification of the choice can be found in section 5.6.

Table 2: Logistic Regression Classifier

Target variable	Precision	% of data points predicted lower than actual values
Job Satisfaction	0.4309	13.6% [Appendix 13.1]
Competence	0.4830	20.4% [Appendix 13.3]

5.3 Random Forest Classifier

A random forest classifier was built for each variable with tuned hyperparameters. The precision for each model with tuned hyperparameters is listed in Table 3 [Appendix 10]. Both models performed better than the baseline models, but the increment in precision for Competence is relatively small.

Table 3: Random Forest Classifier

Target variable	Precision	% of data points predicted lower than actual values
Job Satisfaction	0.4343	18.9% [Appendix 13.2]
Competence	0.4787	20.7% [Appendix 13.4]

5.4 Decision Tree Classifier

A random forest classifier was built for each variable with tuned hyperparameters. The precision for each model with tuned hyperparameters is listed in Table 4 [Appendix 11]. Both models performed better than the baseline models, but the increment in precision for Competence is relatively small.

Table 4: Decision Tree Classifier

Target variable	Precision	% of data points predicted lower than actual values
Job Satisfaction	0.4343	18.4% [Appendix 13.5]
Competence	0.4749	20.8% [Appendix 13.6]

5.5 K-Nearest Neighbors Classifier (k = 500)

A K-Nearest Neighbors Classifier (KNN) with k = 500 was built for each variable. The precision for each model is listed in Table 5 [Appendix 12]. Even though the k value we used is already large, the precision for each model barely outperformed the baseline model. Therefore, KNN did not perform well in our dataset.

Table 5: KNN Classifier

Target variable	Precision	% of data points predicted lower than actual values
Job Satisfaction	0.3654	17.8% [Appendix 13.7]
Competence	0.4748	20.3% [Appendix 13.8]

5.6 Final Model Selection

Since we were dealing with a multi-class classification problem, precision served as the most important metric to evaluate model performance. However, the precision across all models were not very high and many candidate models had very close precision. Therefore, we need another metric to compare our models. Further analysis of low model precision can be found in section 7.0.

Simply looking at accuracy, recall or MAE did not provide us with too many insights on assessing model performance since they were difficult to interpret when there were 5 classes. Therefore, we analyzed the confusion matrices of all the models and calculated the proportion of data points that were predicted lower than true values [Appendix 13]. Since we wanted a more conservative predictor, models with a higher proportion of underestimated data points were desired.

To summarize, we wanted to select the model with highest precision; if the precision of alternative models were close, the model that was biased to predict lower than true values was selected. The selected model for each target variable is shown in table 6.

Table 6: Final Model Selection

Target variable	Best performing model

Job Satisfaction	Random Forest Classifier
Competence	Logistic Regression Classifier

6.0 Simulation

6.1 Job Performance Optimization Model

We first proposed a job performance optimization model which maximizes a weighted sum of job satisfaction and competence. In our case, the top 25 candidates would be selected from a pool of 200 applicants. The objective function and constraint are shown below to better illustrate.

$$Maximize \sum_{i=1}^{200} (s_i + c_i)x_i \qquad i = 1...200$$

$$subject\ to: \sum_{i=1}^{200} x_i \leq 25 \qquad Maximum\ number\ of\ candidates\ constraint$$

Where

x_i : *Binary variables, Whether to select candidate j, 1 if selected, 0 if not*

s_i : *Predicted job satisfaction*

c_i : *Predicted competence*

Therefore, we generated a sample of size 200 from the testing dataset, the sum of job performance scores of the optimal list of 25 candidates would be generated through the implementation of optimization. [Appendix 14.1]

6.2 Simulation Logistic

We used our selected models to generate the sum of job performance scores of the best 25 candidates based on the same sample of testing dataset as the optimization model.

In order to show how our model can outperform a decision making who guesses how to select people, we then made a random selection model that randomly selected 25 candidates from the same applicant pool as the optimization model, and generated the sum of job performance scores of the 25 candidates following the same procedure.

To evaluate the performance of our model, we compared our model score with the optimal score and the random search score. Simulation method was employed to ensure the consistency and reliability of our model. [Appendix 14.2] We did 100 simulations that each time a sample of size 200 from the testing data set was generated, the optimized scores, predicted scores and random selected scores were also generated for comparison.

6.3 Results

Through 100 replications the average of the predicted sum scores is higher than that of the random selection, and the predictive model beat random selection model 94% of the time, indicating there were only 6 times the random selection result had a higher scores than predicted result [Appendix 14.2].

Table 7: Average sum scores of 25 candidates' job performance(Sum) in 100 replications

	Optimized result	Predicted result	Random selection
Avg. Sum of job performance Scores	182.98	156.71	135.99

(25 candidates)			

6.4 Sensitivity analysis

Although our model presents a good performance according to the result, there are still some factors that need to be considered such as the constraints. We found that having different sets of constraints could affect the overall results. For example, if we want to include another constraint such that at least 5 female candidates among the 25 candidates, the result might become infeasible since it's very likely that the sample data set with a size of 200 contained less than 5 females. That's because the total number of female respondents is less than 10% of total males in the whole data set.

We also changed the objective function to maximize the product of job satisfaction and competence to make a further comparison [Appendix 14.3]. Under this condition, through 100 replications, our predictive model beat random selection model 100% of the time. Compared with the sum of scores result table above, we found that random selection model did not increase too much, while the optimized scores and predicted scores were both significantly improved. This indicates that multiplication model can magnify the score differences, therefore was more biased to select applicants that had both high Job Satisfaction and high Competence than the original model.

Table 8: Average sum scores of 25 candidates' job performance(Multiplication) in 100 replications

	Optimized result	Predicted result	Random selection
Avg. Sum of job performance Scores (25 candidates)	355.92	261.83	186.17

7.0 Discussion

7.1 Comparison and possible restrictions of the job satisfaction predictor

From EDA we know that even though Job Satisfaction has 5 classes, 66% of the population belong to class 3 and 4. Our best model performed 8% better than the baseline model in terms of precision thus had much space for improvement. Most importantly, from table 3 we can tell 37.7% of the data points were still predicted higher than the true values, indicating that even the most conservative model tended to predict higher.

7.2 Comparison and possible restrictions of the competence predictor

Similar to Job Satisfaction, Competence also has 5 classes and an even more imbalanced dataset. Almost 48% of all data belong to class 3. Therefore, the baseline model had a noticeably high precision, and the best model only performed 1% better than the baseline model. From table 2 we know that 31.3% of the data points were still predicted higher than the true values, meaning this model also tended to predict higher.

7.3 Comparison and possible restrictions of job performance optimization model

Through sensitivity analysis, we found out that our Job Satisfaction optimization model is highly sensitive to changes in constraints and objective function coefficient. Also, since both Job Satisfaction and Competence are not continuous and tended to be overestimated, oftentimes there were more than 25 applicants had equal Job Performance scores. Therefore, we needed to include more constraints to filter out more applicants, but adding more constraints led to unstable optimization model performance.

7.4 Possible factors not considered

The entire Job Performance simulation model is based on the assumption that the survey responses reflect reality. However, the Competence distribution shows that almost 70% of the respondents thought that they were above average, which is not realistic.

In addition, there might be other features in the dataset that correlate with Job Performance well such as career aspiration [Appendix 1]. Using a combination of more variables to predict Job performance during simulation might be helpful to counteract the biases in each individual base model.

8.0 Further Direction

8.1 Future Research Direction

In the future, the team is interested in further investigating key job performance indicators of IT industry practitioners. Aside from Job Satisfaction and Competence, more features will also be considered in the job performance simulation model to reduce the impact of data imbalance of each individual variable. We will attempt to re-sample the data to increase the proportion of minority classes, thus reducing the tendency to overestimate. Finally, we will further explore how to reduce the sensitivity of the simulation model thereby allowing hiring managers to customize their recruitment constraints without impairing the model performance.

9.0 References:

[1] J. Meister, “Ten HR Trends In The Age Of Artificial Intelligence,” *Forbes*, 09-Jan-2019. [Online]. Available: https://www.forbes.com/sites/jeannemeister/2019/01/08/ten-hr-trends-in-the-age-of-artificial-intelligence/#72bf39223219. [Accessed: 06-Dec-2019].

[2].ti-avatar Tom E. Jones, T. E. Jones, A. DuVernet, S. Gallo, A. Price, and Training Industry Logo Emblem AvatarDefault Training Industry Avatar, “Key Factors That Influence Job Performance: The Performance Pathway Model,” *Training Industry*, 06-Sep-2019. [Online]. Available: https://trainingindustry.com/articles/performance-management/key-factors-that-influence-job-performance-the-performance-pathway-model/. [Accessed: 06-Dec-2019].

Appendix:

Appendix 1. Eliminated data dictionary

Eliminated feature	**Definition**
Respondent	Randomized respondent ID number
LastInt	What you were asked to do in the most recent successful job interview
FizzBuzz	1 if you have been asked to solve FizzBuzz in an interview, 0 if not
CurrencySymbol	Type of currency used in day-to-day
CurrencyDesc	Type of currency used in day-to-day
CompTotal	Total compensation of currency
CompFreq	Weekly, monthly, or yearly currency compensation
CodeRev	1 if review code as a work, 0 if not
CodeRevHrs	Number of hours on reviewing code per week
UnitTests	1 if company employs unit tests in the developing of products, 0 if not
PurchaseHow	How does your company make technology-related decisions
DevEnviron	Which development environment(s) do you use regularly
OpSys	What is the primary operating system in which you work?
Containers	How do you use containers
BlockchainOrg	How is your organization thinking about or implementing blockchain technology
OpenSourcer	How often do you contribute to open source?
MgrMoney	1 if you believe that you need to be a manager to make more money, 0 if not
BlockchainIs	What is Blockchain / cryptocurrency technology
BetterLife	1 if you think people born today will have a better life than their parents, 0 if not
ITperson	1 if you are the "IT support person" for your family, 0 if not
OffOn	1 if you have tried turning it off and on again, 0 if not
Extraversion	1 if you prefer online chat or IRL conversations, 0 if not
ScreenName	What do you call it
SOVisit1st	When did you first visit Stack Overflow

SOVisitFreq	How frequently would you say you visit Stack Overflow
SOVisitTo	The purpose of visiting Stack Overflow
SOFindAnswer	how many times a week do you find (and use) an answer on Stack Overflow
SOTimeSaved	What is your last time to solve a coding problem using Stack Overflow,
SOHowMuchTime	How much time did you save
SOAccount	1 if you have a Stack Overflow account, 0 if not
SOPartFreq	How frequently would you say you participate in Q&A on Stack Overflow
SOJobs	Have you ever used or visited Stack Overflow Jobs
OpenSource	How do you feel about the quality of open source software
MgrWant	1 if you want to become a manager yourself in the future, 0 if not
EntTeams	1 if you ever used Stack Overflow for Enterprise or Stack Overflow for Teams, 0 if not
SOComm	1 if you consider yourself a member of the Stack Overflow community, 0 if not
WelcomeChange	How welcome do you feel on Stack Overflow
SONewContent	1 if you like to see any of the following on Stack Overflow, 0 if not
Trans	1 if you identify as transgender, 0 if not
Sexuality	What sexual do you currently identify as
SurveyLength	How do you feel about the length of the survey this year
SurveyEase	How easy or difficult was this survey to complete
ResumeUpdate	What is the primary reason that you updated your resume
DatabaseWorkedWith	What database environments have you done extensive development work in over the past year, and which do you want to work in over the next year
PlatformWorkedWith	What platforms have you done extensive development work for over the past year
WebFrameWorkedWith	What web frameworks have you done extensive development work in over the past year, and which do you want to work in over the next year
EduOther	What types of non-degree education have you used or participated in
Student	1 if you currently enrolled in a formal, degree-granting college or university program, 0 if not
YearsCode	Number of years have you been coding
MiscTechWorkedWith	What other frameworks, libraries, and tools have you done extensive development work in over the past year, and which do you want to work in over the next year
Dependents	1 if you have any dependents (e.g., children, elders, or others) that you care for, 0 if not
SocialMedia	What social media site do you use the most
LanguageDesireNextYear	What programming, scripting, and markup languages have you done extensive development work in over the past year, and which do you want to work in over the next year
DatabaseDesireNextYear	What database environments have you done extensive development work in over the past year, and which do you want to work in over the next year
PlatformDesireNextYear	What platforms have you done extensive development work for over the past year
WebFrameDesireNextYear	What web frameworks have you done extensive development work in over the past year, and which do you want to work in over the next year

MiscTechDesireNextYear	What other frameworks, libraries, and tools have you done extensive development work in over the past year, and which do you want to work in over the next year
CareerSat	How satisfied are you with your career thus far
LastHireDate	When was the last time you took a job with a new employer
JobSeek	Which best describes your current job-seeking status
WorkRemote	How often do you work remotely
MainBranch	Which options best describes you today
WorkChallenge	what are your greatest challenges to productivity as a developer

Appendix 2. Adopted Data dictionary

Feature	**Definition**
Employment	current employment status
Country	country of residence
EdLevel	completed highest level of formal education
Orgsize	number of workers in the organization/company
Age1stCode	age of writing the first line of code
ImpSyn	rate of the level of competence
InfluComp	level of influence/power in the company when making technology-related decisions
Age	age (in years)
Gender	gender
BED	Developer back-end
FED	Developer front-end
Hobbyist	1 if coding is worker's hobby, 0 if not
YearsCodePro	number of years coding professionally
JobSat	level of satisfaction to current job
ConfiManager	Level of confidence in manager's capability
ConvertedComp	salary converted to annual USD salaries
WorkWeekHrs	number of working hours per week
Workplan	Level of working structure
WorkLoc	preferred places to work
Ethnicity	Descent
DBA	Data or business analyst
DSML	Data scientist or machine learning specialist
FSD	Full-stack developer
DE	Data engineer
OrgDiversity	Diversity of the company or organization
FlexSchedule	Diversity of the company or organization

OrgFinancial	Financial performance or funding status of the company or organization
Remote	Remote work options
WorkImpact	How widely used or impactful my work output would be
OfficeEnv	Office environment or company culture
ProDevelop	Opportunities for professional development
C	Programming language(1 if the respondent has this skill)
C#	Programming language(1 if the respondent has this skill)
Clojure	Programming language(1 if the respondent has this skill)
Elixir	Programming language(1 if the respondent has this skill)
C++	Programming language(1 if the respondent has this skill)
Kotlin	Programming language(1 if the respondent has this skill)
Scala	Programming language(1 if the respondent has this skill)
VBA	Programming language(1 if the respondent has this skill)
HTML/CSS	Programming language(1 if the respondent has this skill)
Java	Programming language(1 if the respondent has this skill)
JavaScript	Programming language(1 if the respondent has this skill)
Python	Programming language(1 if the respondent has this skill)
R	Programming language(1 if the respondent has this skill)
SQL	Programming language(1 if the respondent has this skill)
Computer Science/Computer Engineering	Field of study(1 if it is the major program for the respondent)
Information System	Field of study(1 if it is the major program for the respondent)
Natural Science	Field of study(1 if it is the major program for the respondent)
Other Engineering Disciplines	Field of study(1 if it is the major program for the respondent)
Web development/design	Field of study(1 if it is the major program for the respondent)
Business	Field of study(1 if it is the major program for the respondent)
Mathematics/Statistics	Field of study(1 if it is the major program for the respondent)
Fine Arts	Field of study(1 if it is the major program for the respondent)
Social Science	Field of study(1 if it is the major program for the respondent)
Humanities	Field of study(1 if it is the major program for the respondent)
Health Science	Field of study(1 if it is the major program for the respondent)

Appendix 3. Parsing Code

```
[ ] #-------------Parse_Step1.-------------------
    def Parse(val):
      counter_list =[]
      new = val.str.split(";",expand = False)
      for k in new:
            counter_list.append([item for i,item in enumerate(k)])
      return counter_list
```

```
[ ] df_copy = df.copy()
```

```
[ ] #-------------Parse_Step2.-------------------
    def ParsePlus(dev,types_interest):
      dfy = pd.DataFrame(columns = types_interest)
      for iter in dev:
        row_to_append = []
        for typ in types_interest:
            if typ in iter:
              row_to_append.append(1)
            else:
              row_to_append.append(0)
        s = pd.Series(row_to_append, index=dfy.columns)
        dfy = dfy.append(s, ignore_index=True)
      return dfy
```

```
[43] #--------------Select options from multiple choice types attributes--------------------------
     # 1. DevType
     types_interested = ['Data or business analyst','Data scientist or machine learning specialist','Developer, back-end',
                         'Developer, front-end','Developer, full-stack','Engineer, data']

     # 2. JobFactor
     factors_interested = ['Diversity of the company or organization','Flex time or a flexible schedule', 'Financial performance or funding status of t
                           'Remote work options','How widely used or impactful my work output would be','Office environment or company culture','Opport

     # 3. LanguageWorkwith
     languages_interested = ['C','C#','Clojure','Elixir','C++','Kotlin','Scala','VBA','HTML/CSS','Java','JavaScript','Python','R','SQL']

     # 4. Platform
     platforms_interested = ['Android', 'AWS', 'Google Cloud Platform', 'Heroku', 'iOS', 'Linux', 'Mac OS', 'Microsoft Azure', 'Raspberry Pi',
                             'Windows', 'Wordpress']
```

```
[ ] #---------------Parsing by using 2 functions above-----------------------
     # 1. Parse DevType
     devt = Parse(df_copy['DevType'])
     df_1 = ParsePlus(devt,types_interested)

     # 2. Parse JobFactor
     df_copy = df_copy.reset_index(drop=True)
     job = Parse(df_copy['JobFactors'])
     df_2 = ParsePlus(job,factors_interested)

     # 3. Parse LanguageWorkwith
     df_copy = df_copy.reset_index(drop=True)
     lang = Parse(df_copy['LanguageWorkedWith'])
     df_3 = ParsePlus(lang,languages_interested)

     # 4. Parse LanguageWorkwith
     df_copy = df_copy.reset_index(drop=True)
     pla = Parse(df_copy['PlatformWorkedWith'])
     df_4 = ParsePlus(pla,platforms_interested)
```

Appendix 4. Impute

```
# median = 57287; median = 6
df['ConvertedComp'].fillna(57287.0, inplace=True)
df['YearsCodePro'].fillna(6.0, inplace=True)
```

Appendix 5. Eliminated Non-Categorical Variables

```
[ ] df.drop(df.index[df['UndergradMajor']=='I never declared a major'],inplace = True)
    df.drop(df.index[df['MgrIdiot']=="I don't have a manager"],inplace = True)
    df = df.drop(df[df.Ethnicity.isin(["I don't know", "I prefer not to answer"])].index)
    df = df.drop(df[df.YearsCodePro.isin(["Less than 1 year","More than 50 years"])].index)
    df = df.drop(df[df.Gender.isin(["Man;Non-binary, genderqueer, or gender non-conforming",
                                    "Woman;Man","Woman;Man;Non-binary, genderqueer, or gender non-conforming",
                                    "Woman;Non-binary, genderqueer, or gender non-conforming",
                                    "Non-binary, genderqueer, or gender non-conforming"])].index)

    print(df.groupby(['Gender']).size())
    print(df.shape)
```

```
Gender
Man      30126
Woman     2317
dtype: int64
(32443, 22)
```

Appendix 6. Converted the Likert-scale Job Satisfaction Question into 5 Categorical Variables

```
[52] new['JobSat'].replace(['Very dissatisfied','Slightly dissatisfied',
                            'Neither satisfied nor dissatisfied','Slightly satisfied',
                            'Very satisfied'],[0,1,2,3,4],inplace=True)
```

```
[ ] new['ImpSyn'].replace(['Far below average','A little below average',
                           'Average','A little above average',
                           'Far above average'],[0,1,2,3,4],inplace=True)
```

Appendix 7. Correlation Output (Descending order):

JobSat	1.000000
ConfiManager	0.392543
InfuComp	0.152445
Office	0.111234
WorkPlan	0.088907
CompanyCulture	0.059883
ConvertedComp	0.045753
Python	0.032405
Woman	0.024092
Natural Science	0.022773
Scala	0.017774
Hobbyist	0.015997
DSML	0.015547
Humanities	0.014542
R	0.014077
Mathematics/Statistics	0.014027
HTML/CSS	0.011941
Social Science	0.011752
DE	0.010887
Fine Arts	0.010754
YearsCodePro	0.010634
Kotlin	0.010112
Health Science	0.009076
Elixir	0.007714
FED	0.006263
WorkImpact	0.006161
Clojure	0.006090
WorkWeekHrs	0.005306
JavaScript	0.004598
OrgDiversity	0.004024
FlexSchedule	0.003404
Business	0.003064
C++	-0.000587
Age	-0.001545
C	-0.003921
Computer Science/Computer Engineering	-0.005572
EdLevel	-0.006406

```
FSD                                    -0.006557
SQL                                    -0.008258
Other Engineering Disciplines          -0.009396
DBA                                    -0.012102
C#                                     -0.013422
BED                                    -0.014883
Information System                     -0.016517
Java                                   -0.017152
OrgFinancial                           -0.017713
VBA                                    -0.021394
Web development/design                 -0.021544
Man                                    -0.024092
Age1stCode                             -0.029314
Remote                                 -0.032997
ProDevelop                             -0.033573
Other places                           -0.072493
Home                                   -0.075196
```

Appendix 8. Baseline Model

```
# the baseline model is a dummy model that always predicts the average jobsat of the training dataset

def dummy_model(y_train,y_test):
  length = len(y_test)
  score = math.ceil(y_train.mean())
  scores = []
  for i in range(length):
    scores.append(score)
  Scores = pd.Series(scores)
  return Scores
```

	precision	recall	f1-score	support
JobSat=0	0.00	0.00	0.00	467
JobSat=1	0.00	0.00	0.00	1010
JobSat=2	0.00	0.00	0.00	689
JobSat=3	0.36	1.00	0.53	2311
JobSat=4	0.00	0.00	0.00	1992
accuracy			0.36	6469
macro avg	0.07	0.20	0.11	6469
weighted avg	0.13	0.36	0.19	6469

Appendix 9. Grid Search for Logistic Regression and results

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
grid={"C": [0.001, 0.01, 0.1, 1, 10], "class_weight": [None, 'balanced'],"solver": ['saga', 'newton-cg']}
logreg=LogisticRegression(multi_class= 'multinomial')
logreg_cv = GridSearchCV(logregC, grid, scoring='precision_micro')
logreg_cv.fit(X_train, Y_train)
```

```
[59] logReg_jobsat= LogisticRegression(multi_class = 'multinomial', solver = 'newton-cg', class_weight = 'balanced')
     logReg_jobsat.fit(X_train, Y_train)
     LR_jobsat=logReg_jobsat.predict(X_test)
     test_score = logReg_jobsat.score(X_test, Y_test)
     print('The the test score is {}'.format(test_score))
```

```
/usr/local/lib/python3.6/dist-packages/scipy/optimize/linesearch.py:466: LineSearchWarning: The line search algo
  warn('The line search algorithm did not converge', LineSearchWarning)
/usr/local/lib/python3.6/dist-packages/scipy/optimize/linesearch.py:314: LineSearchWarning: The line search algo
  warn('The line search algorithm did not converge', LineSearchWarning)
The the test score is 0.4309295088621296
```

```
logReg_comp= LogisticRegression(multi_class = 'multinomial', solver = 'newton-cg')
logReg_comp.fit(X_train1,y_train1)
#LR_comp=logReg_comp.predict(X_test1)
train_score8 = logReg_comp.score(X_train1, y_train1)
test_score8 = logReg_comp.score(X_test1, y_test1)
print('The train score is {} and the test score is {}'.format(train_score8, test_score8))
```

```
/usr/local/lib/python3.6/dist-packages/scipy/optimize/linesearch.py:466: LineSearchWarning: The line search algorithm did not converge
  warn('The line search algorithm did not converge', LineSearchWarning)
/usr/local/lib/python3.6/dist-packages/scipy/optimize/linesearch.py:314: LineSearchWarning: The line search algorithm did not converge
  warn('The line search algorithm did not converge', LineSearchWarning)
The train score is 0.4825953113900071 and the test score is 0.4830198890542552
/usr/local/lib/python3.6/dist-packages/sklearn/utils/optimize.py:203: ConvergenceWarning: newton-cg failed to converge. Increase the num
  "number of iterations.", ConvergenceWarning)
```

Appendix 10. Grid Search for Random Forest Classifier and results

Appendix 10.1 Grid search for Job Satisfaction

```
#---------------Grid search for RF JobSat-------------------------------------
#from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint as sp_randint
from scipy.stats import uniform as sp_randFloat

clf = RandomForestClassifier(n_estimators = 100)
param_dist = {"max_depth": sp_randint(1, 20),
              "max_features": sp_randFloat(),
              "min_samples_split": sp_randint(2, 20),
              "bootstrap": [True, False],
              "criterion": ["gini", "entropy"],
             "min_samples_leaf": sp_randint(1, 20),
             "class_weight": [None, "balanced"],
            "max_leaf_nodes": sp_randint(2, 20)
             }

rand = GridSearchCV(clf,param_dist,cv = 5, scoring = 'precision_micro', iid=False)
rand.fit(X_train, y_train)
```

```
print(rand.best_params_)
print(rand.best_score_)
```

```
{'bootstrap': False, 'class_weight': None, 'criterion': 'entropy', 'max_depth': 4, 'max_features': 0.9567111777129871, 'max_leaf_nodes':
0.4383473381153749
```

```
cl4 = RandomForestClassifier(max_depth = 4, max_features = 0.9567111777129871, n_estimators = 100,
                             criterion = 'entropy',bootstrap= False, min_samples_split = 10, max_leaf_nodes = 9,min_samples_leaf = 19)

cl4.fit(X_train, y_train)
train_score4 = cl4.score(X_train, y_train)
test_score4 = cl4.score(X_test, y_test)
print('The train score is {} and the test score is {}'.format(train_score4, test_score4)) # test score = precision score
```

```
The train score is 0.4386184499847772 and the test score is 0.43431200108239754
```

Appendix 10.2 Grid search for Competence

```
[72]
     clf1 = RandomForestClassifier(n_estimators = 100)
     param_dist1 = {"max_depth": sp_randint(1, 20),
                    "max_features": sp_randFloat(),
                    "min_samples_split": sp_randint(2, 20),
                    "bootstrap": [True, False],
                    "criterion": ["gini", "entropy"],
                   "min_samples_leaf": sp_randint(1, 20),
                   "class_weight": [None, "balanced"],
                  "max_leaf_nodes": sp_randint(2, 20)
                   }

     rand1 = GridSearchCV(clf1,param_dist1,cv = 5, scoring = 'precision_micro', iid=False)
     rand1.fit(X_train1, y_train1)
```

```
print(rand1.best_params_)
print(rand1.best_score_)
```

```
{'bootstrap': True, 'class_weight': None, 'criterion': 'gini', 'max_depth': 12, 'max_features': 0.8600290513942017, 'max_leaf_nodes': 16
0.47995661151872265
```

```
cl1 = RandomForestClassifier(random_state = 0, max_depth = 12, max_features = 0.8600290513942017, n_estimators = 100,
                             criterion = 'gini',bootstrap= False, min_samples_split = 16, max_leaf_nodes = 16,min_samples_leaf = 1)
cl1.fit(X_train1, y_train1)
#cl1 = cl.predict(X_test)
train_score1 = cl1.score(X_train1, y_train1)
test_score1 = cl1.score(X_test1, y_test1)
print('The train score is {} and the test score is {}'.format(train_score1, test_score1)) # test score = precision score
```

```
The train score is 0.48164811745204833 and the test score is 0.47869029901231225
```

Appendix 11. Grid Search for Decision Tree and results

Appendix 11.1 Grid Search for Job Satisfaction

```
[81] from sklearn.tree import DecisionTreeClassifier
     from sklearn.model_selection import GridSearchCV

     def dtree_grid_search(X,y,nfolds):
         #create a dictionary of all values we want to test
         param_grid = { 'criterion':['gini','entropy'],'max_depth': np.arange(3, 15)}
         # decision tree model
         dtree_model=DecisionTreeClassifier()
         #use gridsearch to test all values
         dtree_gscv = GridSearchCV(dtree_model, param_grid, cv=nfolds)
         #fit model to data
         dtree_gscv.fit(X, y)
         return dtree_gscv.best_params_
```

```
print(dtree_grid_search(X_train, Y_train, 5))
```

```
{'criterion': 'entropy', 'max_depth': 3}
```

Appendix 11.2 Grid Search for Competence

```
[88] print(dtree_grid_search(x_train, y_train, 5)) #Decision tree for Competence
```

```
{'criterion': 'entropy', 'max_depth': 4}
```

Appendix 11.3 Decision Tree for Job Satisfaction

```
# training a DescisionTreeClassifier
from sklearn.tree import DecisionTreeClassifier
dtree_js = DecisionTreeClassifier(max_depth = 3, criterion='entropy', random_state=1).fit(X_train, Y_train)
js_pred = dtree_js.predict(X_test)
accuracy = dtree_js.score(X_test, Y_test)
print(accuracy)
```

```
0.43431200108239754
```

Appendix 11.4 Decision Tree for Competence

```
# training a DescisionTreeClassifier
from sklearn.tree import DecisionTreeClassifier
dtree_c = DecisionTreeClassifier(max_depth = 4, criterion='entropy', random_state=1).fit(x_train, y_train)
c_pred = dtree_c.predict(x_test)
accuracy = dtree_c.score(x_test, y_test)
print(accuracy)
```

```
0.47490190772561225
```

Appendix 12. KNN Classifier and results

Appendix 12.1 KNN for Competence

```
[55] # training a KNN classifier
     from sklearn.neighbors import KNeighborsClassifier
     knn = KNeighborsClassifier(n_neighbors = 500).fit(x_train, y_train)

     # accuracy on X_test
     accuracy = knn.score(x_test, y_test)

     print(accuracy)
```

```
0.47476660803680154
```

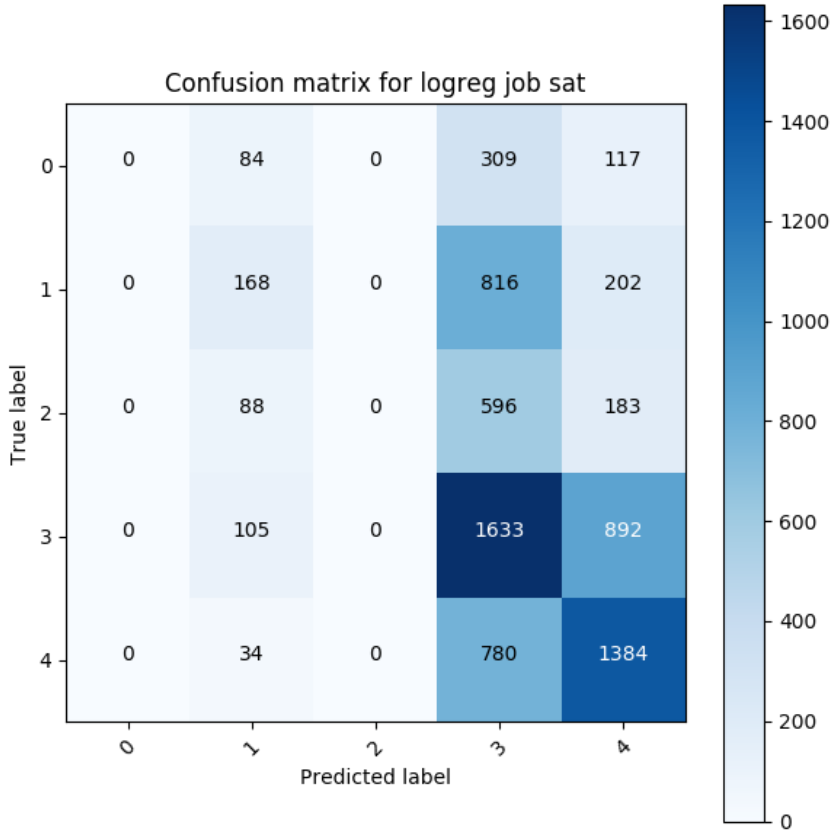
Appendix 12.2 KNN for Job Satisfaction

```
# training a KNN classifier
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors = 500).fit(X_train, Y_train)

# accuracy on X_test
accuracy = knn.score(X_test, Y_test)

print(accuracy)
```

```
0.3654444594777432
```

Appendix 13: Confusion Matrices

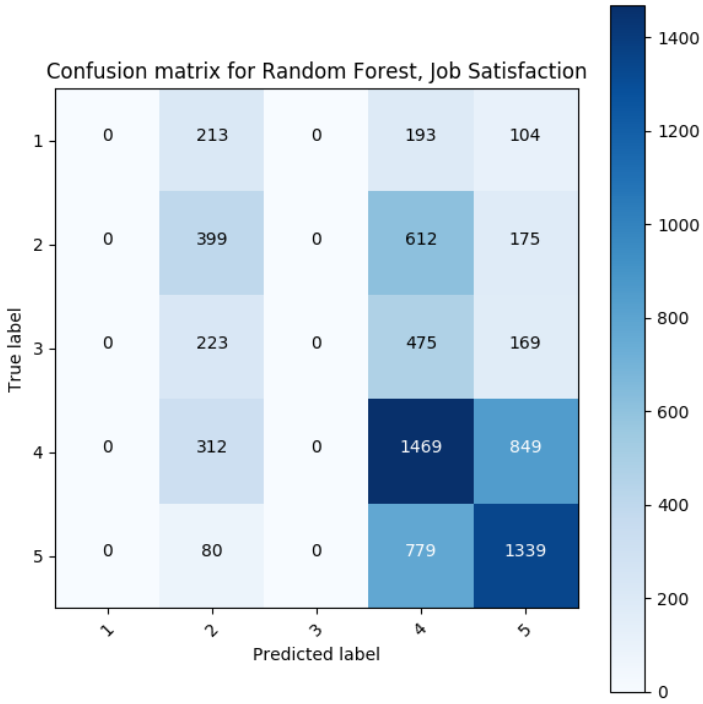
Appendix 13.1 Confusion matrix for Logistic Regression (Job Satisfaction)



Count predicted < true: 780+88+105+34 = 1007

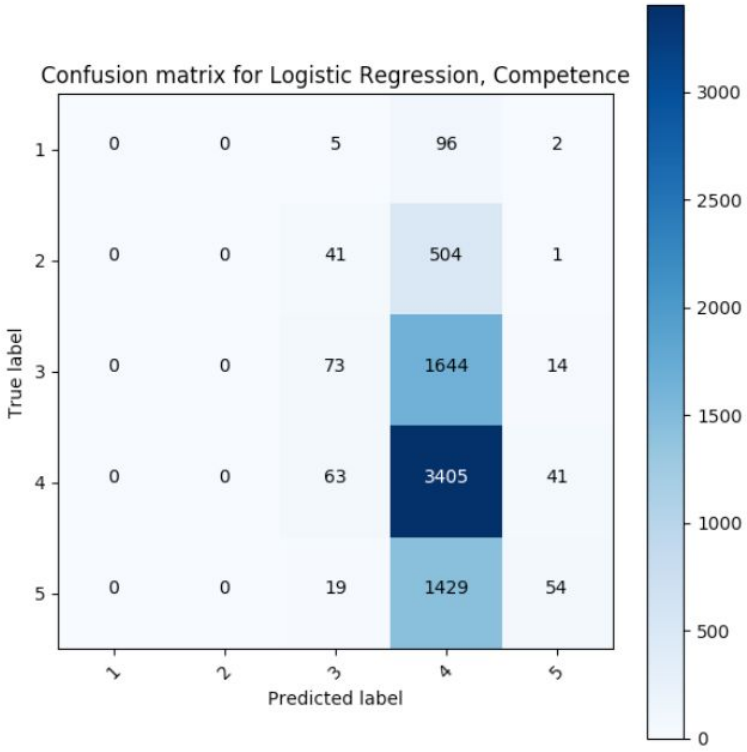
1007/7391 = 13.6%

Appendix 13.2 Confusion matrix for Random Forest (Job Satisfaction)



Count predicted < true: 779+80+312+223= 1394
1394/7391 = 18.9%

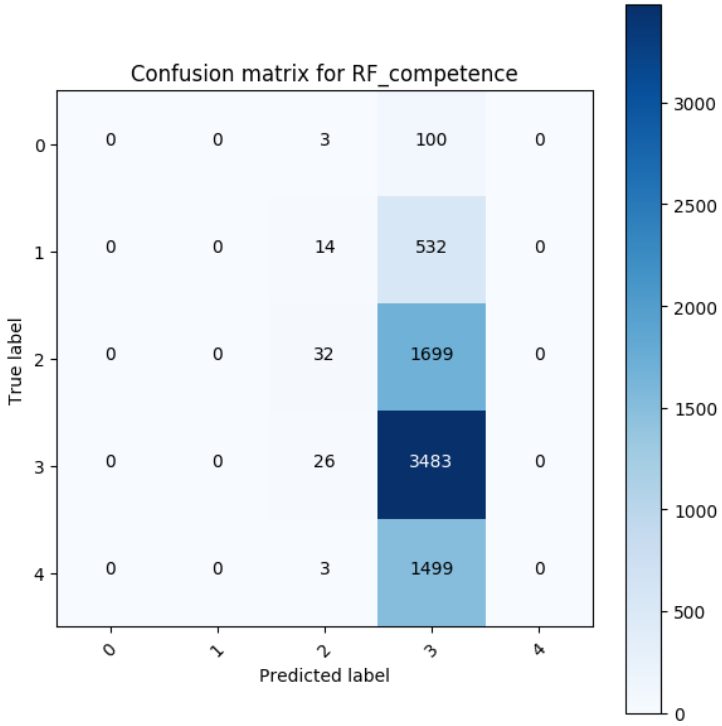
Appendix 13.3 Confusion Matrix for logistic Regression (Competence)



Count predict < true: 1429+19+63=1511

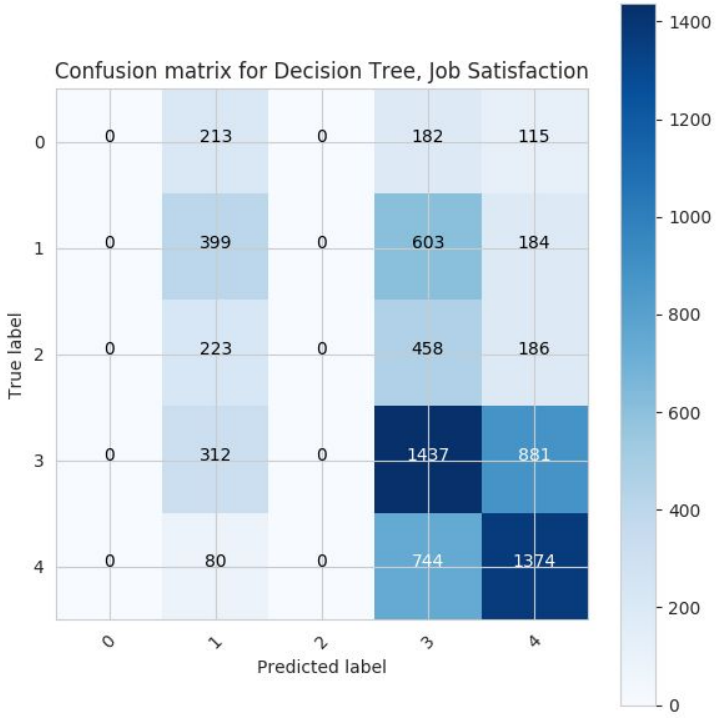
1511/7391 = 20.4%

Appendix 13.4 Confusion Matrix for Random Forest (Competence)



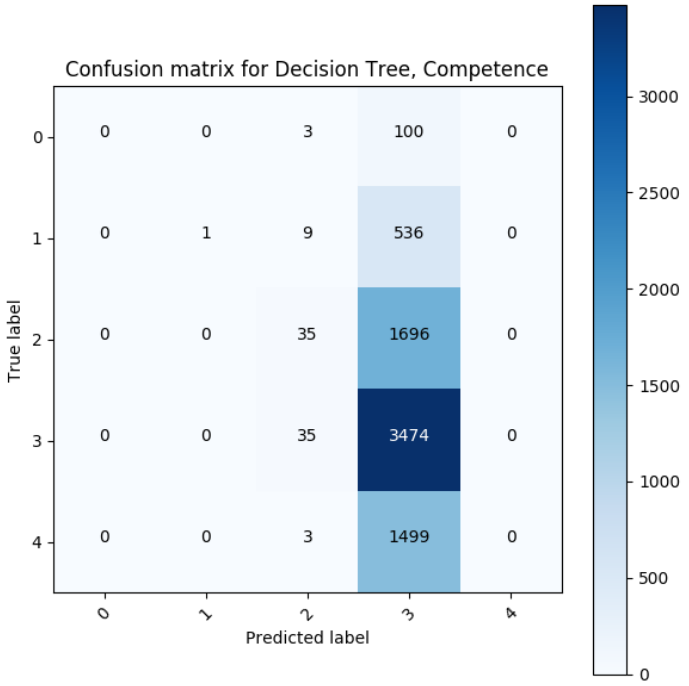
Count predicted < true: 1499+3+26=1528
1528/7391 = 20.7%

Appendix 13.5 Confusion Matrix for Decision Tree (Job Satisfaction)



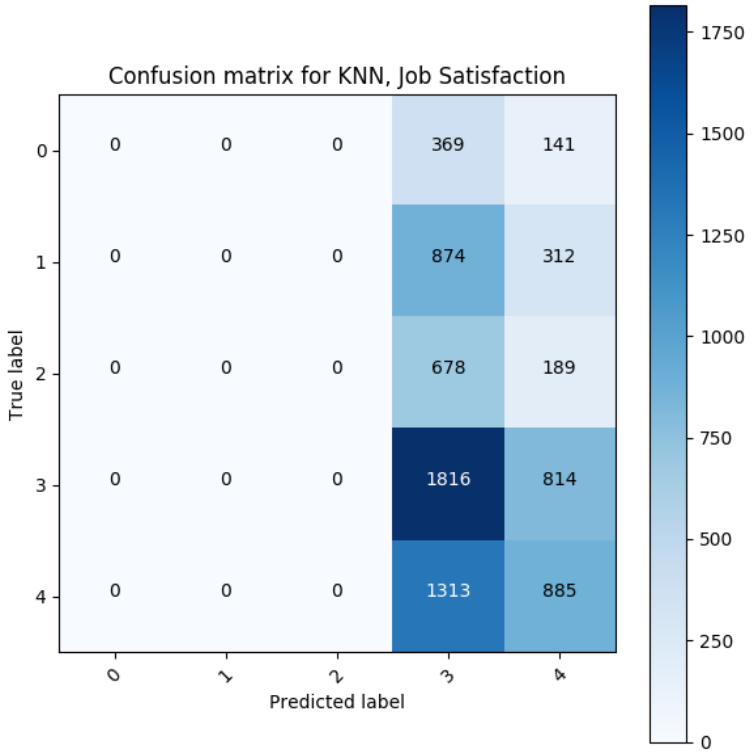
Count predicted < true: 744+80+312+223 = 1359
1359/7391 = 18.4%

Appendix 13.6 Confusion Matrix for Decision Tree (Competence)



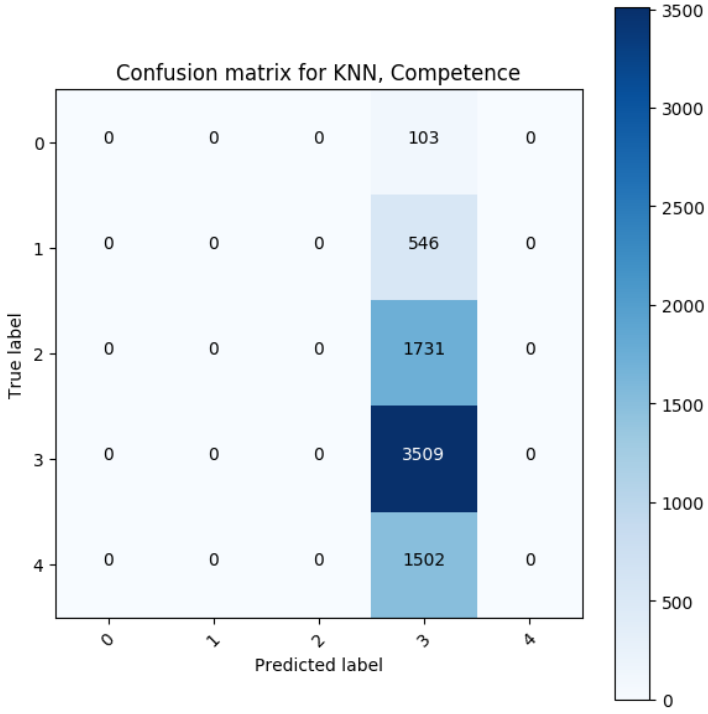
Count predicted < true: 1499 + 3 + 35 = 1537
1537/7391 = 20.8%

Appendix 13.7 Confusion Matrix for KNN (Job Satisfaction)



Count predicted < true: 1313
1313/7391 = 17.8%

Appendix 13.8 Confusion Matrix for KNN (Competence)



Count predicted < true: 1502
1502/7391 = 20.3%

Appendix 14. Simulation

Appendix 14.1 Optimization model and predicted model

```
def CheatsDf1(data,Vars):
    #copyy = copied.copy()
   # data = copyy.sample(n=num)
    #Vars = cvx.Variable(len(data),boolean=True)
    Coef1 = data.JobSat   #value coefficient for our ***objective function***
    Coef2 = data.ImpSyn
    obj = cvx.Maximize(cvx.sum(Vars*(Coef1+Coef2)))
    cons=[]
    total = cvx.sum(Vars)
    #---------------female-----------------------
    femaleCoef = np.where(data.Woman==1,1,0)
    #---------------python-----------------------
    codeCoef1 = np.where(data.Python == 1,1,0)
    codeCoef2 = np.where(data.Java == 1,1,0)
    #----------------Age----------------------
    ageCoef = data.Age
    #--------------Budget-----------------
    budgCoef = data.ConvertedComp
    #cons.append(femaleCoef*Vars >= 2)
    #cons.append(cvx.sum((codeCoef1+codeCoef2)*Vars) >= 2)
    #cons.append(cvx.sum(Vars*budgCoef)/25 <= 200000)
    #cons.append(cvx.sum(Vars*ageCoef)/25 <= 50)
    cons.append(total <= 25)
    prob = cvx.Problem(obj,cons)
    return prob
```

```
def PredictDf(data):
    Vars = cvx.Variable(len(data),boolean=True)
    Coef1 = cl4.predict(data)
    Coef2 = logReg_comp.predict(data)
    obj1 = cvx.Maximize(cvx.sum(Vars*(Coef1+Coef2)))
    cons=[]
    total = cvx.sum(Vars)
    #---------------female-----------------------
    femaleCoef = np.where(data.Woman==1,1,0)
    #---------------python----------------------
    #codeCoef1 = np.where(data.Python == 1,1,0)
    #codeCoef2 = np.where(data.Java == 1,1,0)
    #----------------Age----------------------
    #ageCoef = data.Age
    #--------------Budget-----------------
    budgCoef = data.ConvertedComp

    cons.append(total <= 25)
    pro = cvx.Problem(obj1,cons)
    return pro
```

Appendix 14.2 Simulation Output

```
#----------------------Usable 02----------------------------
res =[]
pre = []
Plist = []
Klist = []
Rlist=[]
num=100
for k in range(num):
  data2 = cop.sample(n=200,replace = True)
  #--------Cheats--------------------
  Vars = cvx.Variable(len(data2),boolean=True)
  p = CheatsDf1(data2,Vars)
  maxval = p.solve(solver='ECOS_BB',verbose=False)
  res.append(int(maxval))
  #objdf1 = data2.iloc[(list(np.flatnonzero(Vars.value > 0.001)))]
  #---------predict-------------------
  data5 = data2.copy().drop(columns=['JobSat','ImpSyn'])
  #ans = predict(data4)
  #pre.append(ans)
  mod = PredictDf(data5)
  lar = mod.solve(solver='ECOS_BB',verbose=False)
  pre.append(int(lar))
  #----------Random----------------------
  data3 = data2.sample(n=25)
  a = data3.JobSat.tolist()
  b = data3.ImpSyn.tolist()
  random=sum(a)+sum(b)
  Rlist.append(random)
print(res)
print(pre)
print(Rlist)
print("Cheats: {},Predict: {}, Random: {}".format(sum(res)/num,sum(pre)/num,sum(Rlist)/num))
```

```
[179, 185, 196, 192, 184, 175, 192, 192, 171, 173, 184, 194, 185, 186, 186, 186, 183, 185, 187, 192, 173, 198, 197, 190, 192, 189, 171,
[157, 156, 163, 167, 168, 149, 167, 158, 152, 144, 161, 168, 167, 154, 165, 157, 162, 168, 153, 170, 142, 166, 163, 166, 161, 160, 143,
[128, 141, 135, 143, 135, 128, 142, 137, 131, 123, 140, 138, 133, 130, 134, 143, 142, 143, 133, 135, 131, 125, 146, 144, 146, 136, 134,
Cheats: 182.98,Predict: 156.71, Random: 135.99
```

Optimal Results:
[179, 185, 196, 192, 184, 175, 192, 192, 171, 173, 184, 194, 185, 186, 186, 186, 183, 185, 187, 192, 173, 198, 197, 190, 192, 189, 171, 190, 184, 168, 195, 189, 186, 184, 190, 175, 153, 177, 184, 170, 168, 188, 195, 181, 195, 188, 185, 195, 194, 191, 187, 189, 187, 165, 180, 187, 177, 168, 182, 167, 195, 200, 185, 182, 181, 164, 179, 196, 184, 183, 167, 168, 174, 184, 193, 185, 161, 192, 189, 186, 183, 158, 174, 185, 195, 176, 194, 189, 183, 196, 191, 187, 194, 174, 184, 161, 186, 142, 192, 175]
Predicted Results:
[157, 156, 163, 167, 168, 149, 167, 158, 152, 144, 161, 168, 167, 154, 165, 157, 162, 168, 153, 170, 142, 166, 163, 166, 161, 160, 143, 164, 146, 153, 159, 159, 157, 158, 166, 145, 136, 150, 155, 136, 144, 164, 163, 160, 167, 166, 162, 158, 160, 162, 165, 160, 162, 145, 156, 160, 157, 143, 160, 141, 165, 165, 166, 161, 159, 142, 152, 165, 160, 155, 151, 144, 153, 166, 164, 160, 137, 163, 164, 163, 154, 130, 151, 150, 163, 149, 166, 163, 158, 171, 160, 162, 157, 148, 161, 135, 163, 126, 158, 145]
Random:
[128, 141, 135, 143, 135, 128, 142, 137, 131, 123, 140, 138, 133, 130, 134, 143, 142, 143, 133, 135, 131, 125, 146, 144, 146, 136, 134, 146, 138, 125, 131, 139, 133, 137, 147, 141, 114, 142, 134, 122, 137, 137, 145, 145, 129, 133, 139, 139, 165, 149, 125, 143, 127, 130, 130, 137, 141, 146, 128, 127, 140, 136, 128, 120, 143, 134, 121, 143, 133, 142, 143, 138, 125, 125, 139, 119, 147, 120, 135, 122, 131, 135, 121, 159, 152, 124, 146, 133, 131, 151, 136, 132, 147, 148, 145, 146, 123, 131, 143, 135]
Cheats: 182.98, Predict: 156.71, Random: 135.99

Appendix 14.3 Result of Multiplication Model

```
res =[]
pre = []
Plist = []
Klist = []
Rlist=[]
ak=0
tk=0
for k in range(100):
  data2 = cop.sample(n=200,replace = True)
  Vars = cvx.Variable(len(data2),boolean=True)
  p = CheatsDf(data2,Vars)
  #-------Cheats--------------------
  maxval = p.solve(solver='ECOS_BB',verbose=False)
  res.append(int(maxval))
  objdf1 = data2.iloc[(list(np.flatnonzero(Vars.value > 0.001)))]
  #---------predict--------------------
  #data4 = objdf1.copy().drop(columns=['JobSat','ImpSyn'])
  data5 = data2.copy().drop(columns=['JobSat','ImpSyn'])
  mod2 = PredictMul(data5)
  lar = mod2.solve(solver='ECOS_BB',verbose=False)
  pre.append(int(lar2))
  #----------Random----------------------
  data3 = data2.sample(n=25)
  a = data3.JobSat
  b = data3.ImpSyn
 # random=numpy.prod(a)numpy.prod(b)
  tk = sum(a*b)
  Rlist.append(tk)
print(res)
print(pre)
print(Rlist)
print("Cheats: {},Predict: {}, Random: {}".format(sum(res)/100,sum(pre)/100,sum(Rlist)/100))
```

```
[344, 380, 344, 352, 388, 360, 348, 380, 344, 396, 328, 348, 356, 364, 372, 368, 336, 340, 328, 356, 348, 356, 356, 384, 344, 336, 368,
[230, 280, 250, 262, 255, 283, 256, 273, 276, 261, 267, 270, 277, 226, 279, 280, 249, 252, 258, 243, 264, 268, 277, 255, 262, 264, 234,
[251, 197, 169, 205, 208, 197, 151, 192, 200, 224, 182, 176, 192, 129, 199, 196, 150, 205, 212, 198, 184, 196, 206, 189, 166, 181, 176,
Cheats: 355.92,Predict: 261.83, Random: 186.17
```